

Builder Portfolio Java Console App Documentation

This document serves as a comprehensive report for the Builder Portfolio Java Console Application, detailing its core functionalities through code snippets and explanatory text. This application is designed to serve several users consisting of different roles and functionalities. This Console app consists of four distinct users: Admin, Project Manager, Builder, Client.

GitHub: https://github.com/kaushal380/Zeta_training_miniproject

User roles and their functionalities

User	Major Functionalities
Admin	→ Manage Projects (Create, update delete) → Manage Users (Add, update, delete) → Assign project to manager
Project Manager	→ Update Project Status → Manage Tasks (Create, update and delete) → Assign Task to builders
Builder	→ Update Task status
Client	→ Purchase a Project → View purchased projects

1. Authentication Dashboard

The Authentication functionality allows the user to login and register. This will validate several cases like the validity of the email and password credentials, duplication of user emails and prompts the user for effective authentication service. In addition, the program checks for input validation for all input types String, Integers, Date, email etc. Upon

successful login or registration of the user, the service will automatically fetch the user and display the corresponding user dashboard in the further workflow.

```
System.out.println("==== BUILDER PORTFOLIO APPLICATION ====");

User user =
    AuthenticationDashboard.authenticate(authService, scanner);

if (user == null) {
    System.out.println("Authentication failed.");
    return;
}

System.out.println("\nLogin Successful!");
System.out.println(user);
```

```
switch (user.getRole()) {

    case ADMIN ->
        new AdminDashboard(projectService, authService, assignmentService, userService, scanner).start(user);

    case PROJECT_MANAGER ->
        new ProjectManagerDashboard(
            projectService,
            taskService,
            assignmentService,
            userService,
            scanner
        ).start(user);

    case BUILDER ->
        BuilderDashboard.start(
            user,
            taskService,
            projectService,
            scanner
        );

    case CLIENT -> new ClientDashboard(projectService, assignmentService, scanner).start(user);

    default ->
        System.out.println("Role not supported.");
}
```

```

Feb 19, 2026 10:08:02 AM repositories.UserRepository loadFromFile
INFO: Loaded 2 users.
===== BUILDER PORTFOLIO APPLICATION =====
=====
1. Login
2. Register
3. Exit
=====

Choose an option: 1

===== LOGIN =====
Email: admin@gmail.com
Password: admin
Welcome back, admin!
Feb 19, 2026 10:08:09 AM services.AuthenticationService login
INFO: Login successful for: admin@gmail.com

Login Successful!
User ID: b7907814-ea27-4ead-ae90-469ba5f0fc75
Name: admin
Email: admin@gmail.com
Role: ADMIN

```

Login

```

Choose an option: 2

===== REGISTER USER =====
Full Name: Kaushal Sambanna
Phone Number: asdfhjw
Enter a valid 10-digit Indian number starting with 6-9.
Phone Number: 6789012312
Email: abcdefg
Enter a valid email (example: abc@xyz.com).
Email: kaushal@gmail.com
Password:
Password cannot be empty.
Password: password123
Date of Birth (YYYY-MM-DD): 20051201
Invalid date format.
Date of Birth (YYYY-MM-DD): 2005-12-01

Enter Address Details
City: Bangalore
State: Karnataka
Zip Code (6 digits): 1d1231
Zip must be 6 digits.
Zip Code (6 digits): 100100
Country: India
Registration successful. Welcome, Kaushal Sambanna!
Feb 19, 2026 10:20:57 AM repositories.UserRepository addUser
INFO: User added to repository: kaushal@gmail.com
Feb 19, 2026 10:20:57 AM services.AuthenticationService register
INFO: User registered successfully with email: kaushal@gmail.com

```

Register

2. Admin Dashboard

This Admin Dashboard serves as the central control panel for managing the Builder Portfolio system. It allows the administrator to create, update, and delete projects, manage user registrations, and assign projects to managers. The dashboard also provides full visibility into all registered users and system activities. It ensures that administrative operations are handled securely and efficiently from a single interface.

```
Login Successful!
User ID: b7907814-ea27-4ead-ae90-469ba5f0fc75
Name: admin
Email: admin@gmail.com
Role: ADMIN
```

```
=====
                ADMIN PANEL
=====
1. View Projects
2. Create Project
3. Update Project
4. Delete Project
5. Register User
6. Assign Project to Manager
7. View All Users
8. Delete User
0. Logout
=====
Select an option: |
```

2.1 Create and delete Project

The Create Project module allows the administrator to add a new project to the system by providing essential details such as name, description, timeline, type, budget, and location. It validates dates and cost to ensure data accuracy before saving the project. The project type categorization helps in organizing different kinds of developments. Once all details are entered correctly, the project is successfully registered in the system. The delete project portal allows the admin to select the desired project and upon successful submission the project will be deleted

CREATE PROJECT

```
private void createProject(User admin) {

    System.out.println("\n===== CREATE PROJECT =====");

    String name = getNonEmptyInput("Project Name");
```

```
String description = getNonEmptyInput("Project Description");

LocalDate startDate = getValidDate("Start Date (YYYY-MM-DD)");

LocalDate endDate;

while (true) {

    endDate = getValidDate("End Date (YYYY-MM-DD)");

    if (endDate.isAfter(startDate)) break;

    System.out.println("End date must be after start date.");

}

ProjectType type = getProjectType();

double cost = getPositiveDouble("Estimated Cost");

System.out.println("\nEnter Project Location Details");

String city = getNonEmptyInput("City");

String state = getNonEmptyInput("State");

String zip = getValidZip();

String country = getNonEmptyInput("Country");

Address location = new Address(city, state, zip, country);

try {

    boolean created = projectService.createProject(
```

```

        admin, name, description, startDate, endDate, type, location,
cost);

        System.out.println(created
                                ? "Project created successfully."
                                : "Project creation failed.");
    } catch (Exception e) {
        System.out.println("Error: " + e.getMessage());
    }
}

```

DELETE PROJECT

```

private void deleteProject(User admin) { 1 usage  Kaushal Sambanna

    List<Project> projectList = new ArrayList<>(projectService.viewAllProjects().values());

    if (projectList.isEmpty()) {
        System.out.println("No projects available.");
        return;
    }

    Project project = selectProject(projectList, action: "DELETE");

    try {
        boolean deleted = projectService.deleteProject(admin, project.getId());
        System.out.println(deleted ? "Project deleted." : "Deletion failed.");
    } catch (Exception e) {
        System.out.println("Error: " + e.getMessage());
    }
}

```

```

===== CREATE PROJECT =====
Project Name: Sattva Knowledge Court
Project Description: A commercial Project in Whitefield
Start Date (YYYY-MM-DD): 2020-05-02
End Date (YYYY-MM-DD): 2027-08-30
Select Project Type:
1. RESIDENTIAL
2. COMMERCIAL
3. GOVERNMENT
4. INDUSTRIAL

2
Estimated Cost: 5000000

Enter Project Location Details
City: Bangalore
State: Karnataka
Zip Code (6 digits): 300077
Country: India
Project created successfully.

```

Create Project

```

=====
ADMIN PANEL
=====
1. View Projects
2. Create Project
3. Update Project
4. Delete Project
5. Register User
6. Assign Project to Manager
7. View All Users
8. Delete User
9. Logout
=====
Select an option: 4

Select project to DELETE:
1. divyasree industrial
2. Aparna Zennith
3. Sattva Knowledge Court
4. satva commercial
Enter number: 4
Project deleted.
Feb 19, 2026 11:19:19 AM services.ProjectService deleteProject
INFO: Project deleted: 68b633d0-851c-4d22-896a-6106e1440b6f

```

Delete Project

2.2 View and Update Project

The Admin Panel provides a centralized interface for managing the entire Builder Portfolio system. Administrators can view all existing projects in a structured list showing status, type, and cost details. The update functionality allows selective modification of project attributes such as name, description, end date, and budget without affecting unchanged fields. It ensures controlled project management while maintaining data integrity. Together, these features enable efficient monitoring and dynamic updating of projects within the system.

VIEW PROJECT

```

private void viewAllProjects() { 1 usage  Kaushal Sambanna

    Map<String, Project> projects = projectService.viewAllProjects();

    if (projects.isEmpty()) {
        System.out.println("\nNo projects found.");
        return;
    }

    System.out.println("\n===== PROJECT LIST =====");

    int index = 1;
    for (Project project : projects.values()) {
        System.out.println(index++ + ". "
            + project.getName()
            + " | " + project.getStatus()
            + " | " + project.getType()
            + " | Cost: " + project.getEstimatedCost());
    }
}

```

UPDATE PROJECT

```

private void updateProject(User admin) {

    List<Project> projectList =

        new ArrayList<>(projectService.viewAllProjects().values());

    if (projectList.isEmpty()) {

        System.out.println("No projects available.");

        return;
    }

    Project project = selectProject(projectList, "UPDATE");
}

```



```
ProjectUpdateRequest request = new ProjectUpdateRequest();

System.out.print("New Name (press Enter to skip): ");

String name = scanner.nextLine().trim();

if (!name.isBlank()) {

    request.setName(name);

}

System.out.print("New Description (press Enter to skip): ");

String desc = scanner.nextLine().trim();

if (!desc.isBlank()) {

    request.setDescription(desc);

}

System.out.print("New End Date (YYYY-MM-DD) (press Enter to skip): ");

String endInput = scanner.nextLine().trim();

if (!endInput.isBlank()) {

    try {

        LocalDate newEndDate = LocalDate.parse(endInput);

        LocalDate referenceStart =

            project.getStartDate();

        if (!newEndDate.isAfter(referenceStart)) {
```

```
        System.out.println("End date must be after start date. Skipping  
update.");  
  
        } else {  
  
            request.setEndDate(newEndDate);  
  
        }  
  
    } catch (Exception e) {  
  
        System.out.println("Invalid date format. Skipping end date  
update.");  
  
    }  
  
}  
  
System.out.print("New Estimated Cost (press Enter to skip): ");  
  
String costInput = scanner.nextLine().trim();  
  
if (!costInput.isBlank()) {  
  
    try {  
  
        double cost = Double.parseDouble(costInput);  
  
        if (cost > 0) {  
  
            request.setEstimatedCost(cost);  
  
        } else {  
  
            System.out.println("Cost must be positive. Skipping update.");  
  
        }  
  
    } catch (Exception e) {  
  
        System.out.println("Invalid cost value. Skipping update.");  
  
    }  
  
}
```

```

try {

    boolean updated =

        projectService.updateProject(admin, project.getId(), request);

    System.out.println(updated

        ? "Project updated successfully."

        : "Update failed.");

} catch (Exception e) {

    System.out.println("Error: " + e.getMessage());

}

}

```

```

=====
                ADMIN PANEL
=====
1. View Projects
2. Create Project
3. Update Project
4. Delete Project
5. Register User
6. Assign Project to Manager
7. View All Users
8. Delete User
9. Logout
=====
Select an option: 1

===== PROJECT LIST =====
1. divyasree industrial | UPCOMING | INDUSTRIAL | Cost: 1000021.0
2. Aparna Zennith | IN_PROGRESS | RESIDENTIAL | Cost: 5000000.0
3. Sattva Knowledge Court | UPCOMING | COMMERCIAL | Cost: 5000000.0
4. satva commercial | COMPLETED | COMMERCIAL | Cost: 3003000.0

```

View Project

```

Select an option: 3

Select project to UPDATE:
1. divyasree industrial
2. Aparna Zennith
3. Sattva Knowledge Court
4. satva commercial
Enter number: 3
New Name (press Enter to skip):
New Description (press Enter to skip):
New End Date (YYYY-MM-DD) (press Enter to skip): 2035-05-05
New Estimated Cost (press Enter to skip):
Project updated successfully.
Feb 19, 2026 11:09:26 AM services.ProjectService updateProject
INFO: Project updated successfully: 035c7ef0-1a31-4810-9b0a-7deb54be6981

```

Update Project

2.3 Assign project to Manager

This feature enables the admin to assign a specific project to a selected project manager in a structured manner. The admin first chooses the project from the available project list, ensuring the correct project is selected. Next, the system displays all available managers, allowing the admin to select the appropriate one. Once confirmed, the project is successfully linked to the chosen manager. A confirmation message and log entry verify that the assignment has been completed.

```
private void assignProject(User admin) {

    List<Project> projectList = new
ArrayList<>(projectService.viewAllProjects().values());

    if (projectList.isEmpty()) {
        System.out.println("No projects available.");
        return;
    }

    Project project = selectProject(projectList, "ASSIGN");

    List<User> managers = userService getUsersByRole(UserRole.PROJECT_MANAGER);

    if (managers.isEmpty()) {
        System.out.println("No managers available.");
        return;
    }

    System.out.println("\nSelect Manager:");
    for (int i = 0; i < managers.size(); i++) {
        System.out.println((i + 1) + ". " + managers.get(i).getName());
    }

    int choice = InputValidator.getValidChoice(scanner, "Enter manager number");

    if (choice < 1 || choice > managers.size()) {
        System.out.println("Invalid selection.");
        return;
    }

    User manager = managers.get(choice - 1);

    try {
        assignmentService.assignProjectToManager(admin, project.getId(),
manager.getId());
        System.out.println("Project assigned successfully.");
    } catch (Exception e) {
        System.out.println("Assignment failed: " + e.getMessage());
    }
}
```

```

=====
ADMIN PANEL
=====
1. View Projects
2. Create Project
3. Update Project
4. Delete Project
5. Register User
6. Assign Project to Manager
7. View All Users
8. Delete User
0. Logout
=====
Select an option: 6

Select project to ASSIGN:
1. divyasree industrial
2. Aparna Zennith
3. Sattva Knowledge Court
4. satva commercial
Enter number: 3

Select Manager:
1. Kaushal
Enter manager number: 1
Project assigned successfully.
Feb 19, 2026 11:14:02 AM services.AssignmentService assignProjectToManager
INF0: Project 035c7ef0-1a31-4810-9b0a-7deb54be6981 assigned to Manager Kaushal

```

2.4 View user and Delete User

This functionality allows the Admin to manage users within the system efficiently. The View All Users option displays a structured list of all registered users along with their unique ID, name, and assigned role. This gives the admin complete visibility over project managers, clients, and other system users. Using the Delete User option, the admin can remove a user by entering their registered email address. Upon successful validation, the system deletes the user and confirms the action. This ensures controlled user management and maintains system integrity through role-based access control.

```

private void viewAllUsers(User admin) {

    List<User> users = userService.getAllUsers(admin);

    if (users.isEmpty()) {
        System.out.println("No users found.");
    }
}

```

```

        return;
    }

    System.out.printf("%-36s | %-20s | %-15s%n", "ID", "Name", "Role");

System.out.println("-----");
System.out.println("-----");

    for (User user : users) {
        System.out.printf("%-36s | %-20s | %-15s%n",
            user.getId(), user.getName(), user.getRole());
    }
}

private void deleteUser(User admin) {

    String email;

    while (true) {
        System.out.print("Enter user email to delete: ");
        email = scanner.nextLine().trim();

        if (email.matches("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\\. [A-Za-z]{2,}$"))
            break;

        System.out.println("Invalid email format.");
    }

    if (!userService.emailExists(email)) {
        System.out.println("No user found with this email.");
        return;
    }

    try {
        userService.deleteUser(email, admin);
        System.out.println("User deleted successfully.");
    } catch (Exception e) {
        System.out.println("Error: " + e.getMessage());
    }
}

```

```

=====
ADMIN PANEL
=====
1. View Projects
2. Create Project
3. Update Project
4. Delete Project
5. Register User
6. Assign Project to Manager
7. View All Users
8. Delete User
0. Logout
=====
Select an option: 7
=====
ID | Name | Role
-----
b7907814-ea27-4ead-ae90-469ba5f0fc75 | Kaushal | PROJECT_MANAGER
b7907814-ea27-4ead-ae90-469ba5f0fc75 | admin | ADMIN
c49ff836-ed28-4810-a21c-4dd6d3715b50 | Kaushal Sambanna | CLIENT

```

View User

```

=====
ADMIN PANEL
=====
1. View Projects
2. Create Project
3. Update Project
4. Delete Project
5. Register User
6. Assign Project to Manager
7. View All Users
8. Delete User
0. Logout
=====
Select an option: 8
Enter user email to delete: kaushal@gmail.com
User deleted successfully.

```

Delete User

3. Project Manager Dashboard

The Project Manager Dashboard provides a centralized interface for managing assigned projects and related tasks. It allows the manager to view their projects, update project details, and modify project status as work progresses. Managers can create, update, delete, and assign tasks to builders to ensure smooth execution. Additionally, they can assign builders to projects and monitor tasks project-wise. This dashboard ensures effective coordination and operational control at the project management level.

```

public void start(User manager) {

    while (true) {

        System.out.println("\n===== PROJECT MANAGER DASHBOARD =====");

        System.out.println("1. View My Projects");

        System.out.println("2. Update Project");

        System.out.println("3. Update Project Status");

        System.out.println("4. Assign Builder to Project");
    }
}

```

```
System.out.println("5. Create Task");

System.out.println("6. Update Task");

System.out.println("7. Delete Task");

System.out.println("8. Assign Task to Builder");

System.out.println("9. View Tasks by Project");

System.out.println("10. Exit");


System.out.print("Choose option: ");

String choice = scanner.nextLine().trim();

try {

    switch (choice) {

        case "1" -> viewMyProjects(manager);

        case "2" -> updateProject(manager);

        case "3" -> updateProjectStatus(manager);

        case "4" -> assignBuilderToProject(manager);

        case "5" -> createTask(manager);

        case "6" -> updateTask(manager);

        case "7" -> deleteTask(manager);

        case "8" -> assignTaskToBuilder(manager);

        case "9" -> viewTasksByProject(manager);

        case "10" -> { return; }

        default -> System.out.println("Invalid choice.");

    }

} catch (Exception e) {

    System.out.println("Operation failed: " + e.getMessage());

}
```



```
}  
  
}  
  
}
```

```
Login Successful!  
User ID: b7907814-ea27-4ead-ae90-469ba5f0fc75  
Name: Kaushal  
Email: test@gmail.com  
Role: PROJECT_MANAGER  
  
===== PROJECT MANAGER DASHBOARD =====  
1. View My Projects  
2. Update Project  
3. Update Project Status  
4. Assign Builder to Project  
5. Create Task  
6. Update Task  
7. Delete Task  
8. Assign Task to Builder  
9. View Tasks by Project  
10. Exit  
Choose option:
```

3.1 View Assigned Project

The “View My Projects” feature allows the Project Manager to see all projects currently assigned to them. It displays a clear, numbered list of project names along with their current status (such as UPCOMING or IN_PROGRESS). This helps the manager quickly understand their responsibilities at a glance. The structured layout makes it easy to select and manage specific projects. Overall, it provides a focused view of only those projects under the manager’s control.

```
private void viewMyProjects(User manager) {  
    List<Project> projects = getManagerProjects(manager);  
  
    if (projects.isEmpty()) {  
        System.out.println("No projects assigned.");  
        return;  
    }  
  
    for (int i = 0; i < projects.size(); i++) {  
        System.out.println((i + 1) + ". " + projects.get(i).getName() + " - " + projects.get(i).getStatus());  
    }  
}
```

```
        + projects.get(i).getName()  
        + " | " + projects.get(i).getStatus();  
    }  
}
```

```
===== PROJECT MANAGER DASHBOARD =====
```

1. View My Projects
2. Update Project
3. Update Project Status
4. Assign Builder to Project
5. Create Task
6. Update Task
7. Delete Task
8. Assign Task to Builder
9. View Tasks by Project
10. Exit

Choose option: **1**

1. divyasree industrial | UPCOMING
2. Sattva Knowledge Court | UPCOMING

3.2 Update Project Status

The "Update Project Status" feature allows the Project Manager to change the current status of a project they are responsible for. After selecting a specific project from their assigned list, the manager is presented with available status options such as UPCOMING, IN_PROGRESS, or COMPLETED. This structured selection ensures only valid transitions

are made. Once confirmed, the system updates the project's status successfully. This helps keep the project lifecycle accurately tracked and up to date.

```
private void updateProjectStatus(User manager) { 1 usage TejasHSBhat +1
    Project project = selectProject(manager);
    if (project == null) return;

    while (true) {
        try {

            ProjectStatus status = getProjectStatus();

            if (status == ProjectStatus.COMPLETED || status == ProjectStatus.UPCOMING) {
                Map<String, Task> viewTasksByProject = taskService.viewTasksByProject(project.getId());
                viewTasksByProject.forEach((String taskid, Task task) -> taskService.deleteTask(manager, taskid));
            }

            projectService.updateProjectStatus(manager, project.getId(), status);
            System.out.println("Status updated.");
            return;

        } catch (Exception e) {
            System.out.println("Invalid status.");
        }
    }
}
```

```
===== PROJECT MANAGER DASHBOARD =====
1. View My Projects
2. Update Project
3. Update Project Status
4. Assign Builder to Project
5. Create Task
6. Update Task
7. Delete Task
8. Assign Task to Builder
9. View Tasks by Project
10. Exit
Choose option: 3
1. divyasree industrial | UPCOMING
2. Sattva Knowledge Court | UPCOMING
Select number: 2
Select Project Status:
1. UPCOMING
2. IN_PROGRESS
3. COMPLETED

2
Status updated.
```

3.3 Assign Project to Builder

The “Assign Builder to Project” feature enables the Project Manager to allocate a specific builder to one of their assigned projects. After selecting the desired project, the manager is presented with a list of available builders to choose from. Upon selection, the system links the builder to the project and confirms the assignment. This ensures clear responsibility distribution and improves task execution within the project. The successful assignment is also logged for tracking and accountability purposes.

```

private void assignBuilderToProject(User manager) { 1 usage  🧑 Kaushal Sambanna
    Project project = selectProject(manager);
    if (project == null) return;

    User builder = selectBuilder();
    if (builder == null) return;

    boolean assigned = assignmentService.assignProjectToBuilder(
        manager, project.getId(), builder.getId()
    );

    System.out.println(assigned ? "Builder assigned." : "Assignment failed.");
}

```

===== PROJECT MANAGER DASHBOARD =====

1. View My Projects
2. Update Project
3. Update Project Status
4. Assign Builder to Project
5. Create Task
6. Update Task
7. Delete Task
8. Assign Task to Builder
9. View Tasks by Project
10. Exit

Choose option: 4

1. divyasree industrial | UPCOMING
2. Sattva Knowledge Court | IN_PROGRESS

Select number: 2

1. builder | ID: b7907814-ea27-4ead-ae90-469ba5f0fc74

Select number: 1

Builder assigned.

Feb 19, 2026 11:51:56 AM services.AssignmentService assignProjectToBuilder
 INFO: Project 035c7ef0-1a31-4810-9b0a-7deb54be6981 assigned to Builder builder

3.4 Create and Update Task

The create task feature allows the Project Manager to create a new task under a selected project. The manager provides task details such as name, description, start date, and end date. The task is then linked to the chosen project and initialized with a default status (e.g., PENDING). This helps in breaking down project work into manageable units and tracking progress effectively. Upon successful creation, the system confirms that the task has been added. Updating Task functionality enables the Project Manager to modify an existing task. After selecting the task from the list, the manager can update specific fields such as the task description without affecting other details. The selective update approach ensures flexibility and prevents unnecessary data changes. Once the update is completed, the system confirms that the task has been successfully modified.

```
private void createTask(User manager) {
    Project project = selectProject(manager);
    if (project == null) return;

    System.out.print("Task Name: ");
    String name = scanner.nextLine().trim();

    System.out.print("Task Description: ");
    String description = scanner.nextLine().trim();

    LocalDate start = getValidDate("Start Date (yyyy-mm-dd): ");

    LocalDate end;
    while (true) {
        end = getValidDate("End Date (yyyy-mm-dd): ");
        if (end.isAfter(start)) break;
        System.out.println("End date must be after start date.");
    }

    if (project.getStatus() == ProjectStatus.COMPLETED) {
        System.out.println("The project is marked completed, cannot create new tasks");
        return;
    } else if (project.getStatus() == ProjectStatus.UPCOMING) {
        project.setStatus(ProjectStatus.IN_PROGRESS);
    }

    boolean created = taskService.createTask(
        manager, name, description, start, end, project.getId()
    );
}
```

```

        System.out.println(created ? "Task created." : "Creation failed.");
    }

    private void updateTask(User manager) {
        Task task = selectTask(manager);
        if (task == null) return;

        TaskUpdateRequest request = new TaskUpdateRequest();

        System.out.print("New Description (Enter to skip): ");
        String desc = scanner.nextLine().trim();
        if (!desc.isBlank()) request.setDescription(desc);

        boolean updated = taskService.updateTask(manager, task.getId(), request);
        System.out.println(updated ? "Task updated." : "Update failed.");
    }
}

```

Create task output

```

===== PROJECT MANAGER DASHBOARD =====
1. View My Projects
2. Update Project
3. Update Project Status
4. Assign Builder to Project
5. Create Task
6. Update Task
7. Delete Task
8. Assign Task to Builder
9. View Tasks by Project
10. Exit
Choose option: 5
1. divyasree industrial | UPCOMING
2. Sattva Knowledge Court | IN_PROGRESS
Select number: 2
Task Name: knowledge court task 1
Task Description: task1 to do the layout
Start Date (yyyy-mm-dd): 2025-12-12
End Date (yyyy-mm-dd): 2026-01-02
Task created.

```

Update Task output

```
===== PROJECT MANAGER DASHBOARD =====
1. View My Projects
2. Update Project
3. Update Project Status
4. Assign Builder to Project
5. Create Task
6. Update Task
7. Delete Task
8. Assign Task to Builder
9. View Tasks by Project
10. Exit
Choose option: 6
1. knowledge court task 1 | Project: Sattva Knowledge Court | Status: PENDING
Select number: 1
New Description (Enter to skip): new task1 for improved layout
Task updated.
```

3.4 Assign task to builder

This feature enables the Project Manager to assign a specific task to a selected builder. The manager first chooses the task from the project's task list and then selects an available builder from the displayed options. Once confirmed, the task is linked to the chosen builder for execution. This ensures clear responsibility, better task tracking, and efficient project workflow management.


```
private void assignTaskToBuilder(User manager) { 1 usage  👤 Kaushal Sambanna +1
    Task task = selectTask(manager);
    if (task == null) return;

    User builder = selectBuilder();
    if (builder == null) return;

    boolean assigned = taskService.assignTaskToBuilder(
        manager, task.getId(), builder.getId()
    );

    if (assigned){
        assignmentService.assignProjectToBuilder(manager, task.getProjectId(), builder.getId());
        System.out.println("Task Assigned");
    }
    else {
        System.out.println("Assignment failed");
    }
}
```

```

===== PROJECT MANAGER DASHBOARD =====
1. View My Projects
2. Update Project
3. Update Project Status
4. Assign Builder to Project
5. Create Task
6. Update Task
7. Delete Task
8. Assign Task to Builder
9. View Tasks by Project
10. Exit
Choose option: 8
1. knowledge court task 1 | Project: Sattva Knowledge Court | Status: PENDING
Select number: 1
1. builder | ID: b7907814-ea27-4ead-ae90-469ba5f0fc74
Select number: 1
Task Assigned

```

4. Builder Dashboard

The Builder Dashboard provides a simple and focused interface for builders to manage their assigned work. Builders can view the projects they are part of and check all tasks allocated to them. They also have the ability to update the status of their tasks as work progresses. This dashboard ensures clear visibility of responsibilities and helps track task completion efficiently.

```

public static void start(User builder,
                        TaskService taskService,
                        ProjectService projectService,
                        Scanner scanner) {

    while (true) {

        System.out.println("\n===== BUILDER DASHBOARD =====");
        System.out.println("1. View My Projects");
        System.out.println("2. View My Tasks");
        System.out.println("3. Update Task Status");
    }
}

```

```

        System.out.println("4. Exit");

        System.out.print("Choose option: ");
        String choice = scanner.nextLine().trim();

        try {
            switch (choice) {
                case "1" -> viewMyProjects(builder, projectService);
                case "2" -> viewMyTasks(builder, taskService);
                case "3" -> updateTaskStatus(builder, taskService, scanner);
                case "4" -> { return; }
                default -> System.out.println("Invalid option.");
            }
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}

```

```

Login Successful!
User ID: b7907814-ea27-4ead-ae90-469ba5f0fc74
Name: builder
Email: builder@gmail.com
Role: BUILDER

===== BUILDER DASHBOARD =====
1. View My Projects
2. View My Tasks
3. Update Task Status
4. Exit
Choose option: |

```

4.1 View Projects the builder is working for

This feature allows the builder to view all projects they are currently assigned to. The system filters projects based on the builder's ID and displays only the relevant ones. If no projects are assigned, an appropriate message is shown. Each listed project includes its name and current status, giving the builder clear visibility of their active work.

```
private static void viewMyProjects(User builder, 1 usage  Kaushal Sambanna +1
                                   ProjectService projectService) {

    List<Project> myProjects = new ArrayList<>();

    for (Project p : projectService.viewAllProjects().values()) {
        if (p.getBuilderIds().contains(builder.getId())) {
            myProjects.add(p);
        }
    }

    if (myProjects.isEmpty()) {
        System.out.println("No projects assigned.");
        return;
    }

    System.out.println("\n==== MY PROJECTS =====");

    for (int i = 0; i < myProjects.size(); i++) {
        Project p = myProjects.get(i);
        System.out.println((i + 1) + ". "
                           + p.getName()
                           + " | Status: " + p.getStatus());
    }
}
```

```
===== BUILDER DASHBOARD =====
```

1. View My Projects
2. View My Tasks
3. Update Task Status
4. Exit

```
Choose option: 1
```

```
===== MY PROJECTS =====
```

1. Sattva Knowledge Court | Status: IN_PROGRESS

4.2 View tasks assigned to builder

This feature allows the builder to view all tasks assigned to them in a clear, numbered list. Each task is displayed along with its current status, helping the builder track pending or ongoing work. It provides quick visibility into individual responsibilities within the assigned projects.

```
private static void viewMyTasks(User builder, 1 usage  Kaushal Sambanna +1
                                TaskService taskService) {

    List<Task> tasks = getMyTasks(builder, taskService);

    if (tasks.isEmpty()) {
        System.out.println("No tasks assigned.");
        return;
    }

    System.out.println("\n===== MY TASKS =====");

    for (int i = 0; i < tasks.size(); i++) {
        Task t = tasks.get(i);
        System.out.println((i + 1) + ". "
                           + t.getDescription()
                           + " | Status: " + t.getStatus());
    }
}
```

```
===== BUILDER DASHBOARD =====
```

1. View My Projects
2. View My Tasks
3. Update Task Status
4. Exit

```
Choose option: 2
```

```
===== MY TASKS =====
```

1. new task1 for improved layout | Status: PENDING

4.3 Update task status

This feature allows the builder to update the status of tasks assigned to them through a simple selection process. The builder first selects a task from their task list and then chooses the appropriate status, such as IN_PROGRESS or COMPLETED. This ensures real-time tracking of task progress within the project. Once updated, the system confirms the change, maintaining accurate and up-to-date workflow management.

```
private static void updateTaskStatus(User builder,
                                     TaskService taskService,
                                     Scanner scanner) {

    List<Task> tasks = getMyTasks(builder, taskService);

    if (tasks.isEmpty()) {
        System.out.println("No tasks available.");
        return;
    }

    System.out.println("\n===== SELECT TASK =====");

    for (int i = 0; i < tasks.size(); i++) {
        Task t = tasks.get(i);
```

```

        System.out.println((i + 1) + ". "
            + t.getDescription()
            + " | Status: " + t.getStatus());
    }

    int index = selectIndex(scanner, tasks.size());

    Task selectedTask = tasks.get(index);

    TaskStatus status = selectStatus(scanner);

    boolean updated = taskService.updateTaskStatus(
        builder,
        selectedTask.getId(),
        status
    );

    if (updated) {
        System.out.println("Task status updated successfully.");
    } else {
        System.out.println("Update failed.");
    }
}

```

===== BUILDER DASHBOARD =====

1. View My Projects
2. View My Tasks
3. Update Task Status
4. Exit

Choose option: 3

===== SELECT TASK =====

1. new task1 for improved layout | Status: PENDING

Select number: 1

Select Status:

1. IN_PROGRESS
2. COMPLETED

Choose option: 2

Task status updated successfully.

5. Client Dashboard

The Client Dashboard provides a simple and intuitive interface for clients to interact with available projects. Clients can view all listed projects along with their details before making a decision. The dashboard also allows them to view projects they have already purchased for easy tracking. Through the purchase option, clients can select and buy one or multiple projects. Overall, it ensures a smooth and organized project browsing and purchasing experience.

```
public void start(User client) { 1 usage  Kaushal Sambanna

    while (true) {

        System.out.println("\n=====");
        System.out.println("          CLIENT DASHBOARD");
        System.out.println("=====");
        System.out.println("1. View All Projects");
        System.out.println("2. View Purchased Projects");
        System.out.println("3. Purchase Project");
        System.out.println("0. Logout");
        System.out.println("=====");

        System.out.print("Enter choice: ");
        String input = scanner.nextLine().trim();

        switch (input) {
            case "1" -> viewAllProjects();
            case "2" -> viewPurchasedProjects(client);
            case "3" -> purchaseProject(client);
            case "0" -> {
                System.out.println("Logging out...");
                return;
            }
            default -> System.out.println("Invalid choice. Try again.");
        }
    }
}
```

```
Login Successful!
User ID: c49ff836-ed28-4810-a21c-4dd6d3715b50
Name: Kaushal Sambanna
Email: kaushal@gmail.com
Role: CLIENT
```

```
=====
                        CLIENT DASHBOARD
=====
1. View All Projects
2. View Purchased Projects
3. Purchase Project
0. Logout
=====
Enter choice: |
```

5.1 View Purchased Projects

The View Purchased Projects feature allows clients to see all the projects they have successfully purchased. It filters projects based on the client's ID and displays only those associated with them. Each project is listed with its name, current status, and project type for clear visibility. If no projects have been purchased, an appropriate message is shown. This helps clients easily track and manage their investments in one place.

```
private void viewPurchasedProjects(User client) {

    List<Project> purchased = new ArrayList<>();

    for (Project project : projectService.viewAllProjects().values()) {
        if (project.getClientIds().contains(client.getId())) {
            purchased.add(project);
        }
    }

    if (purchased.isEmpty()) {
```

```

        System.out.println("You have not purchased any projects yet.");
        return;
    }

    System.out.println("\n===== YOUR PROJECTS =====");

    int index = 1;
    for (Project project : purchased) {
        System.out.println(index + ". "
            + project.getName()
            + " | Status: " + project.getStatus()
            + " | Type: " + project.getType());
        index++;
    }

    System.out.println("=====");
}

```

```

=====
                CLIENT DASHBOARD
=====
1. View All Projects
2. View Purchased Projects
3. Purchase Project
0. Logout
=====
Enter choice: 2
You have not purchased any projects yet.

```

```

=====
                CLIENT DASHBOARD
=====
1. View All Projects
2. View Purchased Projects
3. Purchase Project
0. Logout
=====
Enter choice: 2

===== YOUR PROJECTS =====
1. Aparna Zennith | Status: IN_PROGRESS | Type: RESIDENTIAL
=====

```

5.2 Purchase Projects

The Purchase Project feature allows clients to browse all projects that they have not yet purchased. It displays each available project with its name, type, and estimated cost for easy comparison. The client selects a project using a numbered option, ensuring a simple and user-friendly selection process. Once confirmed, the system assigns the project to the client and records the purchase. A success or failure message is displayed to provide clear feedback on the transaction.

```
private void purchaseProject(User client) {

    List<Project> available = new ArrayList<>();

    for (Project project : projectService.viewAllProjects().values()) {
        if (!project.getClientIds().contains(client.getId())) {
            available.add(project);
        }
    }

    if (available.isEmpty()) {
        System.out.println("No projects available for purchase.");
        return;
    }

    System.out.println("\n===== AVAILABLE PROJECTS =====");

    for (int i = 0; i < available.size(); i++) {
        Project p = available.get(i);
        System.out.println((i + 1) + ". "
            + p.getName()
            + " | Type: " + p.getType()
            + " | Cost: " + p.getEstimatedCost());
    }

    int choice;

    while (true) {
        System.out.print("Select project number to purchase: ");
        String input = scanner.nextLine().trim();

        if (input.matches("\\d+")) {
            choice = Integer.parseInt(input);
            if (choice >= 1 && choice <= available.size()) {
                break;
            }
        }

        System.out.println("Invalid selection. Try again.");
    }
}
```

```
}

Project selectedProject = available.get(choice - 1);

try {
    boolean purchased = assignmentService
        .purchaseProject(client, selectedProject.getId());

    if (purchased) {
        System.out.println("Project purchased successfully!");
    } else {
        System.out.println("Purchase failed.");
    }
}

} catch (Exception e) {
    System.out.println("Error: " + e.getMessage());
}

}
```

```

=====
CLIENT DASHBOARD
=====
1. View All Projects
2. View Purchased Projects
3. Purchase Project
0. Logout
=====
Enter choice: 3

===== AVAILABLE PROJECTS =====
1. divyasree industrial | Type: INDUSTRIAL | Cost: 1000021.0
2. Aparna Zennith | Type: RESIDENTIAL | Cost: 5000000.0
3. Sattva Knowledge Court | Type: COMMERCIAL | Cost: 5000000.0
Select project number to purchase: 2
Project purchased successfully!
Feb 19, 2026 12:19:13 PM services.AssignmentService purchaseProject
INFO: Client Kaushal Sambanna purchased project Aparna Zennith

```

6. Test Cases and Code Coverage

The test suite execution demonstrates strong reliability, with all 90 test cases passing successfully. The coverage report shows 100% class coverage (11/11 classes), ensuring every component in the application is tested. Method coverage stands at 95%, indicating that almost all business logic paths are validated through unit tests. Line coverage reaches 90% overall, with repositories achieving up to 95–100% coverage and service layers maintaining high coverage levels. Branch coverage is 87% overall, reflecting thorough validation of conditional logic, especially in repositories (up to 100%) and core services. Overall, the results indicate a well-tested system with comprehensive validation across repositories and services, ensuring robustness, maintainability, and high code quality.

CoverMain xAllTestsSuite x

Stop 'AllTestsSuite' ⌘F5

1 sec 190 ms

Tests passed: 90 of 90 tests – 1 sec 190 ms

TestUserRepository332 ms

- testGetAllUsers()278 ms
- testSaveToFileFailure()2 ms
- testGetUser()3 ms
- testLoadFromFileWhenFileIsEmpty()1 ms
- testLoadFromFileCorruptedJson()15 ms
- testAddUserSuccess()3 ms
- testLoadFromFileWhenFileExists()26 ms
- testAddUserDuplicate()3 ms
- testDefaultConstructor()1 ms

> TestAuthenticationService16 ms

TestProjectService744 ms

/Library/Java/JavaVirtualMachines/temurin-17.jdk/Contents/Home/bin/java ...

Feb 19, 2026 12:27:06 PM repositories.UserRepository loadFromFile

INFO: File not found or empty. Starting clean.

Feb 19, 2026 12:27:06 PM repositories.UserRepository addUser

INFO: User added to repository: one@gmail.com

Feb 19, 2026 12:27:06 PM repositories.UserRepository addUser

INFO: User added to repository: two@gmail.com

Feb 19, 2026 12:27:06 PM repositories.UserRepository loadFromFile

INFO: File not found or empty. Starting clean.

Feb 19, 2026 12:27:06 PM repositories.UserRepository loadFromFile

INFO: File not found or empty. Starting clean.

Feb 19, 2026 12:27:06 PM repositories.UserRepository saveToFile

SEVERE: Failed to save users to file: /Users/.../...

CoverageAllTestsSuite x

Element ^

	Class, %	Method, %	Line, %	Branch, %
all	100% (11/11)	95% (62/65)	90% (324/359)	87% (134/154)
repositories	100% (6/6)	93% (30/32)	95% (130/136)	97% (33/34)
ProjectRepository	100% (2/2)	90% (10/11)	93% (43/46)	100% (10/10)
TaskRepository	100% (2/2)	91% (11/12)	94% (52/55)	94% (17/18)
UserRepository	100% (2/2)	100% (9/9)	100% (35/35)	100% (6/6)
services	100% (5/5)	96% (32/33)	86% (194/223)	84% (101/120)
AssignmentService	100% (1/1)	83% (5/6)	66% (42/63)	70% (24/34)
AuthenticationService	100% (1/1)	100% (4/4)	100% (24/24)	100% (6/6)
ProjectService	100% (1/1)	100% (8/8)	90% (50/55)	85% (29/34)
TaskService	100% (1/1)	100% (9/9)	94% (51/54)	86% (26/30)
UserService	100% (1/1)	100% (6/6)	100% (27/27)	100% (16/16)